

Graphs - II

Breadth-first search → used to search in a graph

Given a graph $G=(V,E)$, and a source vertex S , BFS systematically explores the edges of G to "discover" every vertex that is reachable from S .

- It computes distance from S to each reachable vertex in G .
- It also produces a breadth-first tree with root S that contain all the reachable vertices.
- For any vertex v reachable from S , the simple path in the breadth-first tree from S to v corresponds to a shortest path from S to v in G .
(i.e, a path containing least number of edges)
- works for both directed & undirected graphs

- BFS algorithm colors each vertex:

WHITE, GRAY, and BLACK

↳ Initially every vertex is WHITE

↳ A discovered vertex becomes GRAY

↳ A GRAY vertex whose edges are explored becomes BLACK.

- Initially all vertices are set to WHITE

- All GRAY vertices are kept in a FIFO queue

- BFS assumes that the given graph $G=(V,E)$ is represented by an Adjacency list, the queue is denoted by Q , and it contains the following attributes to v :

↳ $v.color$: WHITE, GRAY, or BLACK

↳ $v.d$ holds the distance from s to v

↳ $v.\pi$ is v 's predecessor in the breadth-first

tree. If v has no predecessor then $v.\pi = NIL$
(v being source or undiscovered)

BFS(G, s)

for each vertex $u \in G.V - \{s\}$

| $u.color = WHITE$
| $u.d = \infty$
| $u.\pi = NIL$

$s.color = GRAY$

$s.d = 0$

$s.\pi = NIL$

$Q = \emptyset$

Enqueue(Q, s)

while $Q \neq \emptyset$ → NIL

| $u = \text{Dequeue}(Q)$

| for each vertex v in $G.Adj[u]$

| | if $v.color == WHITE$

| | | $v.color = GRAY$

| | | $v.d = u.d + 1$

| | | $v.\pi = u$

| | | Enqueue(Q, v)

| $u.color = BLACK$

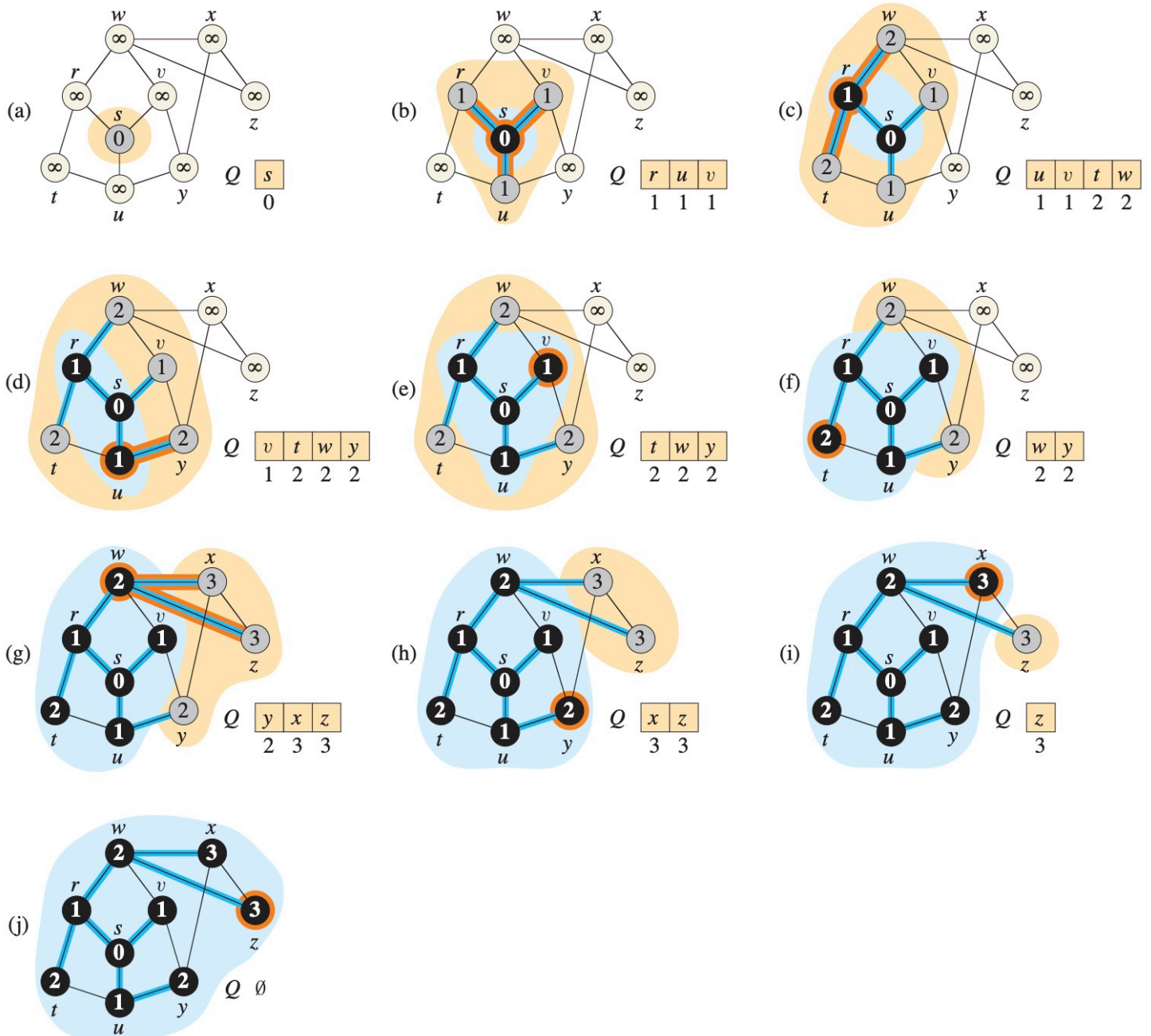
Analysis:

→ Enqueue, Dequeue $\rightarrow O(1)$

→ Total time for queue ops. $\rightarrow O(V)$

→ Since sum of all $|V|$ adjacency lists is $\Theta(|E|)$, and overhead is $O(V)$, Total complexity = $O(|V| + |E|)$

Example:



Depth First Search :

- Searches deeper in the graph whenever possible.
- Explores edges out of the most recently discovered vertex v that still has undiscovered edges leaving it.
- Once all of v 's edges have been explored, the search backtracks to explore edges leaving the vertex from which v was discovered.

Predecessor subgraph: $G_\pi = (V, E_\pi)$, where

$$E_\pi = \{ (v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL} \}$$

- Depth-first search produces a depth-first forest comprising of several depth-first trees
- Edges in E_π are tree-edges

→ Depth-first search also timestamps each vertex
each vertex has two timestamps

① $V.d$ $\Rightarrow$ when V is first discovered

→ Grayed out

② $V.f$ $\Rightarrow$ when search finishes examining
 V 's adjacency list

→ Blacked out

for every vertex u , $u.d < u.f$

- Before time $u.d \rightarrow$ vertex u is WHITE
- Between $u.d$ and $u.f \rightarrow$ vertex u is GRAY
- After time $u.f \rightarrow$ vertex u is BLACK

DFS (G)

for each vertex $u \in G.V$

| $u.color = WHITE$
| $u.\pi = NIL$

time = 0 // global variable

for each vertex $u \in G.V$

| if $u.color == WHITE$
| | DFS-VISIT (G, u)

DFS - VISIT (G, u)

time = time + 1

$u.d = time$ // vertex u is discovered

$u.color = GRAY$

for each vertex v in $G.Adj[u]$

| if $v.color == WHITE$

| | $v.\pi = u$

| | DFS-VISIT (G, v)

time = time + 1

$u.f = time$

$u.color = BLACK$

- Every call to $\text{DFS-VISIT}(n, u)$, makes u the root of a new tree in the depth-first forest.
- Result of depth-first search may depend on the order of selecting vertex u for v & on the order of vertices in the adjacency list.
- Complexity : $\Theta(|V| + |E|)$